COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structure and Algorithm	**SECTION:**	CPE21S2
DATE PERFORMED:	09/18/26	**DATE SUBMITTED:**	09/25/26
NAME:	Nieles, Cenndy M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. _9.1__ Tree ADT

1. PROCEDURE OUTPUT

What procedures did you perform?

Code

Welcome | main.cpp | main2.cpp

main2.cpp > main()

```cpp
#include <iostream>
#include <vector>
#include <string>

struct TreeNode {
    char val;
    std::vector<TreeNode*> children;

    TreeNode(char data) : val(data) {}
};

// Helper function to add a child
void addChild(TreeNode* parent, TreeNode* child) {
    if (parent != nullptr && child != nullptr) {
        parent->children.push_back(child);
    }
}
```

```
18
19  int main() {
20
21      // Level 0 (Root)
22      TreeNode* nodeA = new TreeNode('A');
23
24      // Level 1
25      TreeNode* nodeB = new TreeNode('B');
26      TreeNode* nodeC = new TreeNode('C');
27      TreeNode* nodeD = new TreeNode('D');
28      TreeNode* nodeE = new TreeNode('E');
29      TreeNode* nodeF = new TreeNode('F');
30      TreeNode* nodeG = new TreeNode('G');
31
32      addChild(nodeA, nodeB);
33      addChild(nodeA, nodeC);
34      addChild(nodeA, nodeD);
35      addChild(nodeA, nodeE);
36      addChild(nodeA, nodeF);
37      addChild(nodeA, nodeG);
38
39      // Level 2 & 3
40
41      TreeNode* nodeH = new TreeNode('H');
42      addChild(nodeB, nodeH);
43
44      TreeNode* nodeI = new TreeNode('I');
45      TreeNode* nodeJ = new TreeNode('J');
46      addChild(nodeD, nodeI);
47      addChild(nodeD, nodeJ);
48
49      TreeNode* nodeP = new TreeNode('P');
50      TreeNode* nodeQ = new TreeNode('Q');
51      addChild(nodeE, nodeP);
52      addChild(nodeE, nodeQ);
```

```
53
54      TreeNode* nodeK = new TreeNode('K');
55      TreeNode* nodeL = new TreeNode('L');
56      TreeNode* nodeM = new TreeNode('M');
57      addChild(nodeF, nodeK);
58      addChild(nodeF, nodeL);
59      addChild(nodeF, nodeM);
60
61      TreeNode* nodeN = new TreeNode('N');
62      addChild(nodeG, nodeN);
63
64      std::cout << "General Tree initialized successfully.\n";
65
66      return 0;
67  }
```

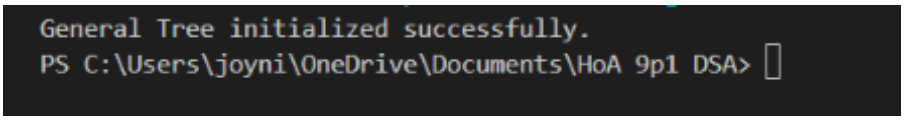
Output	
Observation	Dynamically allocating nodes on the heap using the new operator enabled the construction of multi-level hierarchies linked together through the addChild helper function. Furthermore, the addChild function properly verified pointer safety by ensuring neither the parent nor the child was null before pushing the child node into the parent's children vector.

TABLE 9-1. IMPLEMENTATION OF GENERAL TREE

Node	Height	Depth
A	3	0
B	0	0
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	1
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

TABLE 9-2. NODE HEIGHT AND DEPTH

Traversal	Path / Order Results
Pre-order	A -> B -> C -> D -> H -> E -> I -> J -> P-> Q-> F-> K-> L-> M-> G-> N
Post-order	B-> C-> H-> D-> I-> P-> Q-> J-> E-> K-> L-> M-> F-> N-> G -> A
In-order	B -> A -> C -> H -> D -> I -> E -> P -> J -> Q -> K -> F-> M -> N-> G

Table 9-3. Manual Traversal Strategies

Code




```cpp
//Traversal Function

void preOrder(TreeNode* root) {
    if (!root) return;

    std::cout << root->val << " ";

    for (TreeNode* child : root->children) {
        preOrder(child);
    }
}

void postOrder(TreeNode* root) {
    if (!root) return;

    for (TreeNode* child : root->children) {
        postOrder(child);
    }

    std::cout << root->val << " ";
}

void inOrder(TreeNode* root) {
    if (!root) return;

    if (root->children.empty()) {
        std::cout << root->val << " ";
        return;
    }

    // Traverse first child
    inOrder(root->children[0]);

    // Visit current node
    std::cout << root->val << " ";

    // Traverse remaining children
    for (size_t i = 1; i < root->children.size(); i++) {
        inOrder(root->children[i]);
    }
}

int main() {

```

Welcome | main.cpp | main2.cpp

main2.cpp > main()

```
66      TreeNode* nodeA = new TreeNode('A');
67
68      // Level 1
69      TreeNode* nodeB = new TreeNode('B');
70      TreeNode* nodeC = new TreeNode('C');
71      TreeNode* nodeD = new TreeNode('D');
72      TreeNode* nodeE = new TreeNode('E');
73      TreeNode* nodeF = new TreeNode('F');
74      TreeNode* nodeG = new TreeNode('G');
75
76      addChild(nodeA, nodeB);
77      addChild(nodeA, nodeC);
78      addChild(nodeA, nodeD);
79      addChild(nodeA, nodeE);
80      addChild(nodeA, nodeF);
81      addChild(nodeA, nodeG);
82
83      // Level 2 & 3
84      TreeNode* nodeH = new TreeNode('H');
85      addChild(nodeB, nodeH);
86
87      TreeNode* nodeI = new TreeNode('I');
88      TreeNode* nodeJ = new TreeNode('J');
89      addChild(nodeD, nodeI);
90      addChild(nodeD, nodeJ);
91
92      TreeNode* nodeP = new TreeNode('P');
93      TreeNode* nodeQ = new TreeNode('Q');
94      addChild(nodeE, nodeP);
95      addChild(nodeE, nodeQ);
96
97      TreeNode* nodeK = new TreeNode('K');
98      TreeNode* nodeL = new TreeNode('L');
99      TreeNode* nodeM = new TreeNode('M');
100     addChild(nodeF, nodeK);
101     addChild(nodeF, nodeL);
102     addChild(nodeF, nodeM);
103
104     TreeNode* nodeN = new TreeNode('N');
105     addChild(nodeG, nodeN);
106
107     std::cout << "General Tree initialized successfully.\n";
108
```

```
108
109     // Printing traversals to console
110
111     std::cout << "Pre-Order: ";
112     preOrder(nodeA);
113     std::cout << "\n";
114
115     std::cout << "Post-Order: ";
116     postOrder(nodeA);
117     std::cout << "\n";
118
119     std::cout << "In-Order: ";
120     inOrder(nodeA);
121     std::cout << "\n";
122
123     return 0;
124 }
```

Output

```
Pre-Order: A B H C D I J E P Q F K L M G N
Post-Order: H B C I J D P Q E K L M F N G A
In-Order: H B A C I D J P E Q K F L M N G
PS C:\Users\joyni\OneDrive\Documents\HoA 9p1 DSA>
```

Observation

The output obtained from these two C++ recursive functions is exactly the same as the manual derivation through pencil and paper from Table 9-3. Pre-order process the parent before going down to its children, hence it is useful for cloning or structuring purposes. Post-order goes through all of the descendants first and then processes the parent node, hence it is useful in bottom-up tasks such as freeing nodes.

Code	

```
// Search Helper Functions & findData

bool searchPreOrder(TreeNode* root, char key) {
    if (!root) return false;

    if (root->val == key) return true;

    for (TreeNode* child : root->children) {
        if (searchPreOrder(child, key))
            return true;
    }

    return false;
}

bool searchPostOrder(TreeNode* root, char key) {
    if (!root) return false;

    for (TreeNode* child : root->children) {
        if (searchPostOrder(child, key))
            return true;
    }

    if (root->val == key)
        return true;

    return false;
}

bool searchInOrder(TreeNode* root, char key) {
    if (!root) return false;

    if (!root->children.empty()) {
        if (searchInOrder(root->children[0], key))
            return true;
    }

    if (root->val == key)
        return true;

    for (size_t i = 1; i < root->children.size(); i++) {
        if (searchInOrder(root->children[i], key))
            return true;
```

```
}

void findData(int choice, char key, TreeNode* root) {
    bool found = false;

    switch (choice) {
        case 1:
            found = searchPreOrder(root, key);
            break;

        case 2:
            found = searchPostOrder(root, key);
            break;

        case 3:
            found = searchInOrder(root, key);
            break;

        default:
            break;
    }

    if (found) {
        cout << key << " was found!" << endl;
    }
}

int main() {

    // Level 0 (Root)
    TreeNode* nodeA = new TreeNode('A');

    // Level 1
    TreeNode* nodeB = new TreeNode('B');
    TreeNode* nodeC = new TreeNode('C');
    TreeNode* nodeD = new TreeNode('D');
    TreeNode* nodeE = new TreeNode('E');
    TreeNode* nodeF = new TreeNode('F');
    TreeNode* nodeG = new TreeNode('G');

    addChild(nodeA, nodeB);
    addChild(nodeA, nodeC);
    addChild(nodeA, nodeD);
```

```
88
89      return false;
90  }
91
92  bool searchInOrder(TreeNode* root, char key) {
93      if (!root) return false;
94
95      if (!root->children.empty()) {
96          if (searchInOrder(root->children[0], key))
97              return true;
98      }
99
100     if (root->val == key)
101         return true;
102
103     for (size_t i = 1; i < root->children.size(); i++) {
104         if (searchInOrder(root->children[i], key))
105             return true;
106     }
107
108     return false;
109 }
110
111 void findData(int choice, char key, TreeNode* root) {
112     bool found = false;
```

```
195
196     // findData Demonstrations
197
198     cout << "findData Execution:\n";
199
200     findData(1, 'H', nodeA);
201     findData(2, 'Q', nodeA);
202     findData(3, 'Z', nodeA);
203
204     return 0;
205 }
```

Output

```
findData Execution:
H was found!
Q was found!
PS C:\Users\joyni\OneDrive\Documents\HoA 9p1 DSA>
```

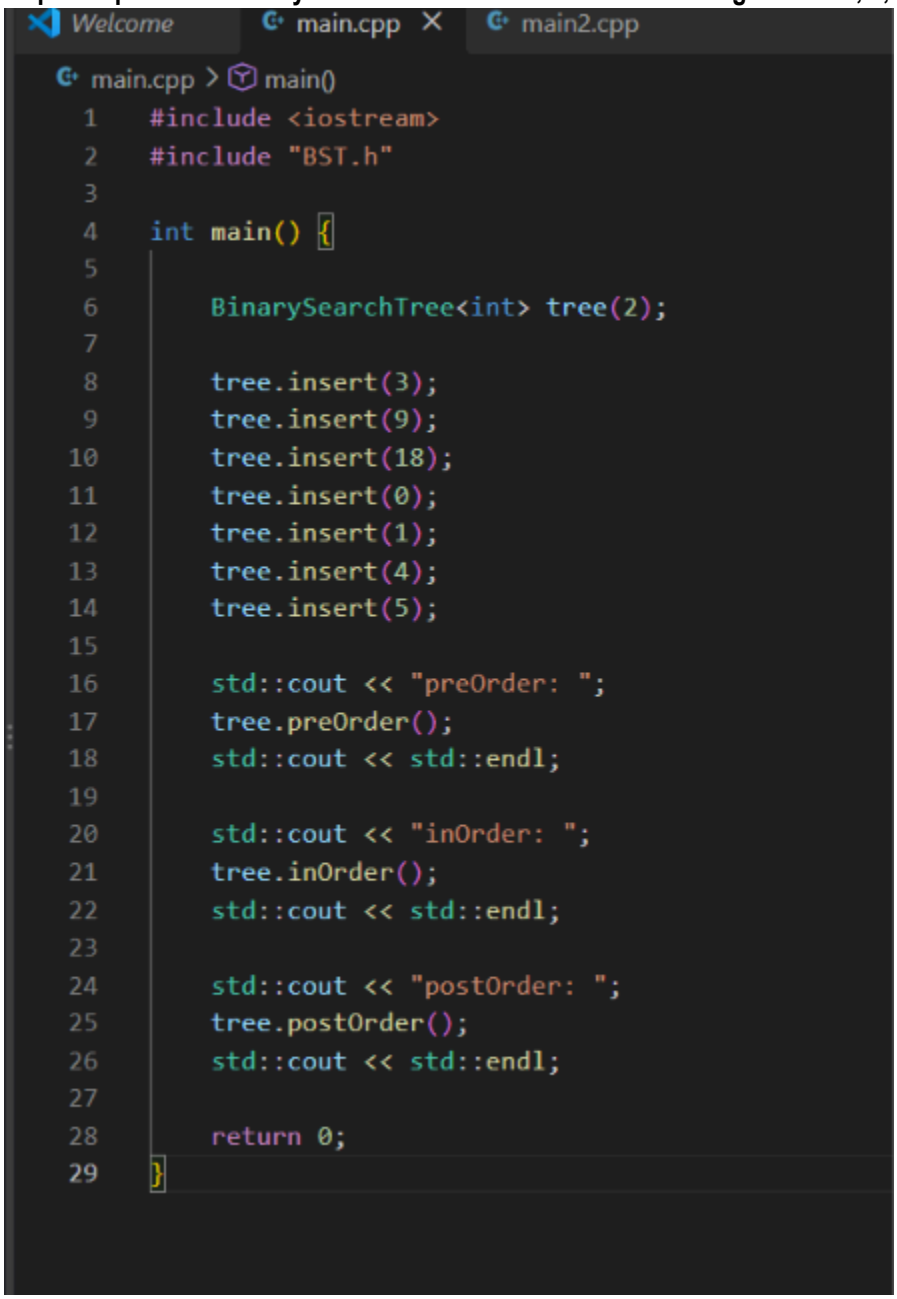
Observation

	using Pre-order, Post-order, or In-order traversal, all the existing nodes within the tree will be examined if the key value is present. The only difference that exists between them is how the nodes will be searched; the search operation on leaf node Q will complete quickly using post-order traversal, while a search on the internal node E will be done quickly using pre-order traversal.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.



main.cpp | BST.h | main2.cpp

BST.h > ...

```h
#ifndef BST_H
#define BST_H

#include <iostream>

template <typename T>
class BinarySearchTree {

private:

    // create a key
    T key;

    // pointers for left and right
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

public:

    // constructor
    BinarySearchTree(T value);

    // insert function
    void insert(T value);

    // traversing
    void preOrder();
    void inOrder();
    void postOrder();

};
```

main.cpp | BST.h | main2.cpp

BST.h > ...

```
1   #ifndef BST_H
33  template <typename T>
34  BinarySearchTree<T>::BinarySearchTree(T value) {
35
36      key = value;
37
38      left = nullptr;
39      right = nullptr;
40  }
41
42  template <typename T>
43  void BinarySearchTree<T>::insert(T value) {
44
45      // whether to insert to the left
46      if (value < key) {
47
48          if (left == nullptr) {
49              left = new BinarySearchTree<T>(value);
50          }
51          else {
52              left->insert(value);
53          }
54      }
55
56      // whether to insert to the right
57      else if (value > key) {
58
59          if (right == nullptr) {
60              right = new BinarySearchTree<T>(value);
61          }
62          else {
63              right->insert(value);
64          }
65      }
66  }
67
```

```cpp
template <typename T>
void BinarySearchTree<T>::preOrder() {

    std::cout << key << " ";

    if (left != nullptr) {
        left->preOrder();
    }

    if (right != nullptr) {
        right->preOrder();
    }
}

template <typename T>
void BinarySearchTree<T>::inOrder() {

    if (left != nullptr) {
        left->inOrder();
    }

    std::cout << key << " ";

    if (right != nullptr) {
        right->inOrder();
    }
}

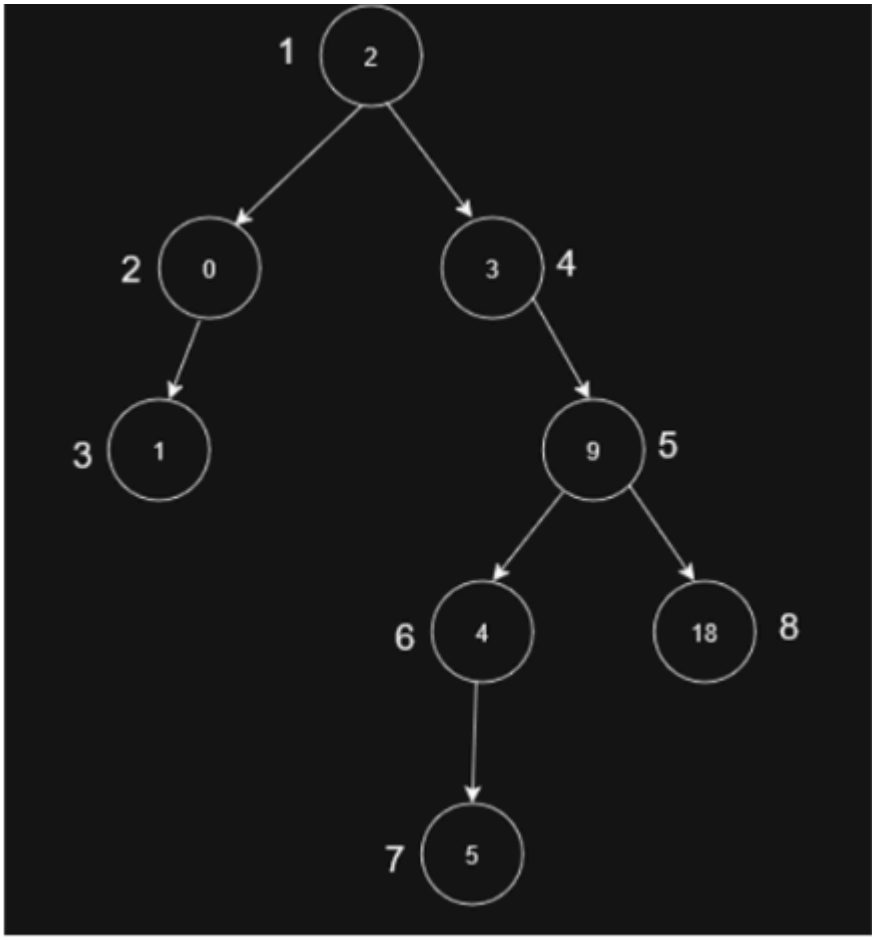
template <typename T>
void BinarySearchTree<T>::postOrder() {

    if (left != nullptr) {
        left->postOrder();
    }
```

```
96  template <typename T>
97  void BinarySearchTree<T>::postOrder() {
98
99      if (left != nullptr) {
100         left->postOrder();
101     }
102
103     if (right != nullptr) {
104         right->postOrder();
105     }
106
107     std::cout << key << " ";
108 }
109
110 #endif
```

BST.h

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.



Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

In-Order traversal was performed using the member function void BinarySearchTree<T>::inOrder() within h=the BinarySearchTree class template.

```
inOrder: 0 1 2 3 4 5 9 18
```

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2

```cpp
void postOrder(BSTNode<T>* node) {
    if (node == nullptr) return;
    postOrder(node->left);
    postOrder(node->right);
    cout << node->data << " ";
}
```

```
Post-order: 1 0 5 4 18 9 3 2
```

```cpp
void postOrder(BSTNode<T>* node) {
    if (node == nullptr) return;
    postOrder(node->left);
    postOrder(node->right);
    cout << node->data << " ";
}
```

```
Post-order: 1 0 5 4 18 9 3 2
```

Traversal Order: Follows a Left -> Right -> Root (bottom-up) sequence.Flow: All child and leaf nodes (e.g., 1, 0, 5, 18) are fully processed before their respective parents (4, 9, 3), ending with the main root node (2).Primary Use Case: This ensures a bottom-up approach, which is ideal for memory freeing since child elements must be deallocated before their parent containers.

In this activity, a General Tree was successfully created using cpp. Different traversal methods Pre-Order, Post-Order, and In-Order were implemented to visit and display the nodes in different sequences. Search functions were also added to locate specific data within the tree. Through this activity, we learned how tree structures organize data hierarchically and how traversal and search algorithms help access information efficiently. Overall, the program demonstrated the basic operations of a General Tree and improved understanding of tree data structures and their applications.Some parts of this activity were difficult for me, especially understanding how the traversal and search functions work in detail. Although I did not fully understand every part of the code yet, I was able to follow the implementation and observe how the program works. With more practice and study, I believe I will gain a deeper understanding of these concepts.Overall, this activity improved my knowledge of tree data structures and strengthened my programming skills in C++. I will continue to review and practice these topics so that I can better understand and apply them in future programming activities.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY